

DAY 1 · 下午 · KNOWLEDGE LAB 整合 · 60 MIN

今天建了一堆文件——它们真的在工作吗？

今天做了什么：DDD + Governance + Capture + Retrieve + Health —— 这些都是「零件」，还没验证「系统」能转。



5 步全绿 = Knowledge 模块在工作。注意：此时还没有 Engine（明天建），agent 会直接写代码——这是预期行为。

一个精心设计的 session——触发所有环节

实验任务（统一下发）

为错题本写一个 recognize 函数：接收一张图片，调用 Bedrock Claude 识别题目内容，返回结构化 JSON。

为什么选这个任务

- 足够复杂——agent 会做出设计决策
- 明确关联 principles——「数据质量」「完成度」都会被触发
- agent 几乎一定会犯错（不加 confidence、不处理异常、不写测试）
- 让你有东西可纠正 → corrections 自然产生

实验流程（每步标注：验证什么）

- | | |
|--------------------------|----------------|
| 1. 开新 session | — |
| 2. 先问「你知道 principles 吗？」 | 验证注入 |
| 3. 下发任务 | — |
| 4. 观察 agent 行为 | 是否受约束 |
| 5. 故意等犯错 → 纠正 | 产生 corrections |
| 6. 完成后结束 session | 触发 capture |
| 7. 检查 corrections.log | 验证落盘 |

Session 启动——agent 知道你的 principles 吗？

● ● ● claude code — new session

you ▶ 在开始工作之前——请告诉我你知道的
关于这个项目的 principles 是什么？

agent ▶ 根据注入的上下文，本项目的 principles:

- 完成 = 主动破坏且失败，而非「没发现问题」
- 数据质量是底线 (confidence 必填)

项目：错题本 · Bedrock · DynamoDB

✅ 预期结果

- agent 能复述你的 3-5 条 principles (措辞略异、核心正确)
- agent 知道项目基本信息 (错题本 + Bedrock + DynamoDB)
- health status 没有红色警报

❌ 如果失败 —— 排查

症状	排查
agent 完全不知道 principles	检查 on-session-start.sh 执行权限 (chmod +x)
知道部分但不完整	检查注入顺序，可能后面内容被截断
health 报红	检查文件是否齐全

下发任务——观察 agent 是否被约束

观察点 (principle 生效的好信号)

- 在设计阶段主动提及你的 principles
- 主动添加 confidence 字段 (数据质量 principle 生效)
- 完成后主动做验证 (完成度 principle 生效)

故意等待的错误 (通常至少 1-2 个)

- 不处理 Bedrock API 异常
- 不验证返回 JSON 的 schema
- 不写测试
- 用 magic number 而非配置
- 错误的并发处理

你的工作

观察

不要提前干预, 让错误发生

等待

记录 agent 犯了什么错

纠正

用自然语言 (「这里不对, 应该...」)

积累

每次纠正都是一条未来的 correction

Session 结束——corrections.log 有内容了吗？

操作

1. 告诉 agent 「任务完成，结束 session」
2. 退出 Claude Code session
3. 等待 SessionEnd hook 执行完（几秒）
4. 检查 corrections.log

log 为空？排查

- SessionEnd hook 是否注册
- hook 脚本执行权限
- Claude CLI 是否可用（`claude --print 测试`）

```
$ cat corrections.log
```

CORRECTION: 没有处理 Bedrock API 异常
(2024-01-15)

CORRECTION: 返回 JSON 没有验证 schema 完整性
(2024-01-15)

DECISION: try-except 包裹整个识别流程，
异常时返回标准错误 (2024-01-15)

DISCOVERY: Bedrock Claude 对数学公式识别率
低于文字 (2024-01-15)

✓ 成功标志：corrections.log 有至少 2 条提取结果 = Knowledge 模块完整闭环

Principles 生效了吗——对比有/无 principles 的行为差异

观察	含义
agent 主动提到了 principles	好——principle 被读到并影响推理
有 principles 注入仍犯相关错误	正常——注入是方向不是硬约束
agent 行为明显受到约束	很好——principle 可操作化成功
principles 100% 失效	写得不够可判定，需要重写

对比实验（时间允许）

不注入 principles 跑同样任务 → 通常差异：有 principles 时 agent 会主动考虑边界、主动写测试。但不是 100%。

教学点：Principles 提供方向，不提供保证

Principles

80% 的时间有效 = 已经很有价值

+ Rules

15% 靠规则提醒兜底

+ Gates

最后 5% 靠门禁物理阻断

三层组合 ≈ 99.5%+ 正确行为

你的 corrections 里有什么——初步 pattern

收集全班 corrections (匿名) → 自然成组

完成度类

- 没写测试
- 测试不充分
- 「完成」太早 (没自验证)

防御性编程类

- 缺少错误处理
- 缺少输入验证

可读性类

- 命名不够清晰
- magic number

对照你的 principles

你的 principles 覆盖了哪些 corrections? 有没有 principles 没预见到的?

未覆盖的 = coverage gap → 需要新 principle 或调整现有 principle。但现在不急——明天跑 SDLC 再积累一天, pattern 会更清晰, 那时做真正的蒸馏。

DAY 1 · 完成

你有了 AgentOS 的大脑

Day 1 成果盘点

- ✓ 理解 AgentOS 全景（两模块 + 基座）
- ✓ 理解 Harness 六大能力
- ✓ Knowledge 骨架：DDD 4 文档 + 三层治理
- ✓ Capture + Retrieve + Health check
- ✓ 端到端验证 Knowledge 循环可运转
- ✓ corrections.log 有真实内容

Knowledge ✓ DONE

Delivery Engine

← 明天

Harness ✓ 已理解

NEXT ► DAY 2

上午

设计 Delivery Engine（Stages → Gates → Profiles）

下午

SDLC 实弹——跑 Engine，Knowledge 实时被填充

明天结束时：Knowledge 从空壳变成有血有肉的。今晚打开 corrections.log，想想「这些背后有几个根因」——明天蒸馏的预热。